LabTest 17/10/2025

Q1. Write a program to find the nth ugly number.

An ugly number is a positive integer whose prime factors are limited to 2, 3, and 5.

Input: A positive integer `n`

Output: The `nth` ugly number

Example 1:

```
Input: n = 10
Output: 12
```

Explanation:
The first 10 ugly numbers are:
`1, 2, 3, 4, 5, 6, 8, 9, 10, 12`
The 10th ugly number is `12`.

Q2. Write a program to merge two binary max heaps into a single max heap.

A binary max heap is a complete binary tree where the value of each node is greater than or equal to the values of its children.

Input:

- Two arrays representing two binary max heaps, `heap1` and `heap2`.
- Each array element is an integer.

Output:

- An array representing the merged max heap.

Example:

```
Input:
heap1 = [10, 5, 6, 2]
heap2 = [12, 7, 9]

Output: [12, 10, 9, 2, 5, 6, 7]
```

Explanation:

- Merge the two heaps into a single array: `[10, 5, 6, 2, 12, 7, 9]`
- Convert the merged array into a max heap using heapify operations.
- The final max heap maintains the property that each parent is greater than its children.

Q3. Write a program to sort an array of integers in ascending order using the Heap Sort algorithm.

Rules:

- You must implement the Heap Sort algorithm yourself.
- Do not use any built-in sorting functions.

- Optimize your heap operations to make the sort as fast as possible.

Input:

- First line: an integer `n` — the size of the array
- Second line: `n` integers separated by spaces — the array to be sorted

Output:

- The sorted array in ascending order, enclosed in square brackets `[]`
- Each number separated by a comma

Example:

Input:

```
6
1 9 3 5 4 7
```

Output:

```
[1,3,4,5,7,9]
```

Explanation:

- Input array: `[1, 9, 3, 5, 4, 7]`
- After applying Heap Sort, the array becomes `[1, 3, 4, 5, 7, 9]`

Q4. Write a program to find the minimum cost to repair roads so that all cities are connected.

- **There are `n` cities and some roads connecting them.**
- **Each road has a repair cost `w`.**
- **After the roads are damaged, you want to repair enough roads to connect all cities with minimum total cost.**

Input:

- A list of edges: `{u, v, w}` where:
  - `u` and `v` are city indices (1-based)
  - `w` is the repair cost of the road connecting `u` and `v`

Output:

- An integer — the minimum total cost to connect all cities

Example:

Input:

```
{{1, 2, 1}, {1, 3, 2}, {1, 4, 3}, {1, 5, 4},
 {2, 3, 5}, {2, 5, 7}, {3, 4, 6}}
Output: 10
```